Program with Output

1.Write a python program to complete the given task using string methods:

a. Find the number of characters in each product name and store it in a list.
b. Convert all product names to uppercase.
c. Find the number of times the word "sale" appears in the product descriptions.
d. Replace all occurrences of the word "red" with "green" in the product descriptions.

Code:

```
# Example data
product_names = ["Product A", "Product B ", "Product C"]
product_descriptions = ["On sale now! Product A is a great product.", "Product B at a discounted price.","Product C in red color is perfect for your needs."]

# Task 1: Find the number of characters in each product name and store it in a list
name_lengths = [len(name) for name in product_names]

# Task 2: Convert all product names to uppercase
uppercase_names = [name.upper() for name in product_names]

# Task 3: Find the number of times the word "sale" appears in the product descriptions
sale_count = 0
for description in product_descriptions:
    sale_count += description.lower().count("sale")

# Task 4: Replace all occurrences of the word "red" with "green" in the product descriptions
updated_descriptions = [description.replace("red", "green") for description in product_descriptions]

# Display the results
print("Number of characters in each product name:", name_lengths)
print("Product names in uppercase:", uppercase_names)
print("Number of times 'sale' appears in product descriptions:", sale_count)
print("Product descriptions with 'red' replaced by 'green':", updated_descriptions)
```

OUTPUT:-

Number of characters in each product name: [9, 10, 9]
Product names in uppercase: ['PRODUCT A', 'PRODUCT B ', 'PRODUCT C']
Number of times 'sale' appears in product descriptions: 1

Product descriptions with 'red' replaced by 'green': ['On sale now! Product A is a great product.', 'Product B at a discounted price.', 'Product C in green color is perfect for your needs.']

2.Write a python program to complete the given task using List and its methods

A. Add five more products to the list and sort them in alphabetical order.
B. Create a new list with the prices of all the products and find the average price.
C. Count the number of products whose price is above 500.
D. Remove the last product from the list and add a new product at the beginning.
E. Find the index of the product with the maximum price and replace its price with a new value.

```
# Initial list of products and prices
products = ["Widget A", "Gadget B", "Thing C", "Device D", "Tool E"]
prices = [300, 450, 600, 700, 800]

# Task 1: Add five more products to the list and sort them in alphabetical order
new_products = ["Item F", "Accessory G", "Product H", "Device I", "Tool J"]
products.extend(new_products)
products.sort()

# Task 2: Create a new list with the prices of all the products and find the average price
all_prices = prices + [350, 500, 650, 750, 900]
average_price = sum(all_prices) / len(all_prices)

# Task 3: Count the number of products whose price is above 500
above_500_count = sum(price > 500 for price in all_prices)

# Task 4: Remove the last product from the list and add a new product at the beginning
last_product = products.pop()
new_first_product = "New Product"
products.insert(0, new_first_product)

# Task 5: Find the index of the product with the maximum price and replace its price with a new
value
max_price_index = prices.index(max(prices))
new_price = 1000
prices[max_price_index] = new_price

# Display the results
print("Products in alphabetical order:", products)
print("Prices of all products:", all_prices)
```

```
print("Average price of all products:", average_price)
print("Number of products above 500:", above_500_count)
print("Updated product list after removing the last product and adding a new one:", products)
print("Prices after replacing the maximum price:", prices)
```

OUTPUT

Products in alphabetical order: ['New Product', 'Accessory G', 'Device D', 'Device I', 'Gadget B', 'Item F', 'Product H', 'Thing C', 'Tool E', 'Tool J']
Prices of all products: [300, 450, 600, 700, 800, 350, 500, 650, 750, 900]
Average price of all products: 600.0
Number of products above 500: 6
Updated product list after removing the last product and adding a new one: ['New Product', 'Accessory G', 'Device D', 'Device I', 'Gadget B', 'Item F', 'Product H', 'Thing C', 'Tool E', 'Tool J']
Prices after replacing the maximum price: [300, 450, 600, 700, 1000]

3.Write a python program to complete the given task using Dictionary and its methods

A. Add three more students to the dictionary with their respective details.
B. Display all the keys in the dictionary.
C. Display all the values in the dictionary.
D. Update the address of one of the students in the dictionary.
E. Remove one of the students from the dictionary.
F. Sort the dictionary by keys and display the sorted dictionary.
G. Sort the dictionary by values and display the sorted dictionary.
H. Find the length of the dictionary.
I. Check if a key is present in the dictionary or not.
J. Clear the dictionary and display it.

```
# Initial dictionary of students
students = {
    'Alice': {'age': 20, 'address': '123 Main St'},
    'Bob': {'age': 22, 'address': '456 Oak Ave'},
    'Charlie': {'age': 21, 'address': '789 Pine Blvd'}
}

# Task 1: Add three more students to the dictionary with their respective details
students['David'] = {'age': 23, 'address': '101 Elm Ln'}
students['Eve'] = {'age': 19, 'address': '202 Birch Dr'}
students['Frank'] = {'age': 25, 'address': '303 Cedar Rd'}

# Task 2: Display all the keys in the dictionary
```

```
print("Keys in the dictionary:", list(students.keys()))

# Task 3: Display all the values in the dictionary
print("Values in the dictionary:", list(students.values()))

# Task 4: Update the address of one of the students in the dictionary
students['Bob']['address'] = '789 Maple Ave'

# Task 5: Remove one of the students from the dictionary
del students['Charlie']

# Task 6: Sort the dictionary by keys and display the sorted dictionary
sorted_by_keys = dict(sorted(students.items()))

# Task 7: Sort the dictionary by values and display the sorted dictionary
sorted_by_values = dict(sorted(students.items(), key=lambda item: item[1]['age']))

# Task 8: Find the length of the dictionary
dictionary_length = len(students)

# Task 9: Check if a key is present in the dictionary or not
key_to_check = 'David'
key_present = key_to_check in students

# Task 10: Clear the dictionary and display it
students.clear()

# Display the results
print("Updated dictionary after adding students:", students)
print("Sorted dictionary by keys:", sorted_by_keys)
print("Sorted dictionary by values:", sorted_by_values)
print("Length of the dictionary:", dictionary_length)
print(f"Is '{key_to_check}' present in the dictionary?: {key_present}")
print("Cleared dictionary:", students)
```

OUTPUT

Keys in the dictionary: ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank']
Values in the dictionary: [{'age': 20, 'address': '123 Main St'}, {'age': 22, 'address': '456 Oak Ave'}, {'age': 21, 'address': '789 Pine Blvd'}, {'age': 23, 'address': '101 Elm Ln'}, {'age': 19, 'address': '202 Birch Dr'}, {'age': 25, 'address': '303 Cedar Rd'}]
Updated dictionary after adding students: {}

Sorted dictionary by keys: {'Alice': {'age': 20, 'address': '123 Main St'}, 'Bob': {'age': 22, 'address': '789 Maple Ave'}, 'David': {'age': 23, 'address': '101 Elm Ln'}, 'Eve': {'age': 19, 'address': '202 Birch Dr'}, 'Frank': {'age': 25, 'address': '303 Cedar Rd'}}
Sorted dictionary by values: {'Eve': {'age': 19, 'address': '202 Birch Dr'}, 'Alice': {'age': 20, 'address': '123 Main St'}, 'Bob': {'age': 22, 'address': '789 Maple Ave'}, 'David': {'age': 23, 'address': '101 Elm Ln'}, 'Frank': {'age': 25, 'address': '303 Cedar Rd'}}
Length of the dictionary: 5
Is 'David' present in the dictionary?: True
Cleared dictionary: {}

4. Create a tuple named products with the following information: product name, product ID, product price, and product quantity. Use the appropriate tuple method to perform the following tasks:

A. Display the entire tuple.
B. Display the length of the tuple.
C. Access the second element of the tuple.
D. Find the index of a particular element in the tuple.
E. Count the number of occurrences of a particular element in the tuple.
F. Concatenate two tuples.
G. Convert the tuple to a list.
H. Check if a particular element is present in the tuple or not.
I. Display the maximum value in the tuple.
J. Display the minimum value in the tuple.

```
# Create a tuple named products with product information: (product name, product ID, product price, product quantity)
products = ("Widget A", 101, 29.99, 50)

# Task 1: Display the entire tuple
print("Products Tuple:", products)

# Task 2: Display the length of the tuple
print("Length of Tuple:", len(products))

# Task 3: Access the second element of the tuple
print("Second Element:", products[1])

# Task 4: Find the index of a particular element in the tuple
element_to_find = 29.99
index_of_element = products.index(element_to_find)
print(f"Index of {element_to_find}: {index_of_element}")

# Task 5: Count the number of occurrences of a particular element in the tuple
occurrences_of_element = products.count(element_to_find)
```

```
print(f"Number of occurrences of {element_to_find}: {occurrences_of_element}")

# Task 6: Concatenate two tuples
additional_products = ("Gadget B", 102, 39.99, 30)
combined_products = products + additional_products
print("Combined Tuples:", combined_products)

# Task 7: Convert the tuple to a list
products_list = list(products)
print("Converted to List:", products_list)

# Task 8: Check if a particular element is present in the tuple or not
element_to_check = 101
is_element_present = element_to_check in products
print(f"Is {element_to_check} present in the tuple?: {is_element_present}")

# Task 9: Display the maximum value in the tuple
max_value = max(products[2], additional_products[2])
print("Maximum Value in Tuple:", max_value)

# Task 10: Display the minimum value in the tuple
min_value = min(products[2], additional_products[2])
print("Minimum Value in Tuple:", min_value)
```

OUTPUT

Products Tuple: ('Widget A', 101, 29.99, 50)
Length of Tuple: 4
Second Element: 101
Index of 29.99: 2
Number of occurrences of 29.99: 1
Combined Tuples: ('Widget A', 101, 29.99, 50, 'Gadget B', 102, 39.99, 30)
Converted to List: ['Widget A', 101, 29.99, 50]
Is 101 present in the tuple?: True
Maximum Value in Tuple: 39.99
Minimum Value in Tuple: 29.99

5. Create a Python program to use a set concept to store the items in your inventory.

Your program should perform the following tasks:

A. Use the add() method to add items to the inventory set.
B. Use the remove() method to remove an item from the inventory set.

C. Use the clear() method to empty the inventory set.
D. Use the update() method to add multiple items to the inventory set.
E. Use the intersection() method to find the common items between two inventory sets.
F. Use the difference() method to find the items that are in one inventory set but not in the other.
G. Use the union() method to combine two inventory sets into one.
H. Use the copy() method to make a copy of the inventory set.
I. Use the pop() method to remove and return an arbitrary item from the inventory set.
J. Use the discard() method to remove an item from the inventory set if it is present, but do not raise an error if the item is not present.

Code:

```
# Create an inventory set
inventory = {"apples", "bananas", "oranges"}

# Add items to the inventory set
inventory.add("grapes")
inventory.add("pears")
print("Inventory after adding items:", inventory)

# Remove an item from the inventory set
inventory.remove("oranges")
print("Inventory after removing an item:", inventory)

# Empty the inventory set
inventory.clear()
print("Inventory after clearing:", inventory)

# Add multiple items to the inventory set
inventory.update(["mangoes", "pineapples"])
print("Inventory after updating:", inventory)

# Create another inventory set for intersection and difference methods
inventory2 = {"apples", "oranges", "peaches"}

# Find the common items between two inventory sets
common_items = inventory.intersection(inventory2)
print("Common items between two inventories:", common_items)

# Find the items that are in one inventory set but not in the other
unique_items = inventory.difference(inventory2)
print("Unique items in inventory 1:", unique_items)
```

```
unique_items = inventory2.difference(inventory)
print("Unique items in inventory 2:", unique_items)

# Combine two inventory sets into one
combined_inventory = inventory.union(inventory2)
print("Combined inventory:", combined_inventory)

# Make a copy of the inventory set
inventory_copy = inventory.copy()
print("Copy of inventory:", inventory_copy)

# Remove and return an arbitrary item from the inventory set
item = inventory.pop()
print("Item removed from inventory:", item)
# Remove an item from the inventory set if it is present
inventory.discard("grapes")
print("Inventory after discarding an item:", inventory)
```

OUTPUT

Inventory after adding items: {'pears', 'bananas', 'oranges', 'grapes', 'apples'}
Inventory after removing an item: {'pears', 'bananas', 'grapes', 'apples'}
Inventory after clearing: set()
Inventory after updating: {'pineapples', 'mangoes'}
Common items between two inventories: set()
Unique items in inventory 1: {'pineapples', 'mangoes'}
Unique items in inventory 2: {'peaches', 'apples', 'oranges'}
Combined inventory: {'peaches', 'mangoes', 'oranges', 'pineapples', 'apples'}
Copy of inventory: {'pineapples', 'mangoes'}
Item removed from inventory: pineapples
Inventory after discarding an item: {'mangoes'}

6. Write a Python program to take input from the user for their grades in five subjects and calculate their overall grade based on the following criteria:

a. Each subject has a weightage of 20% towards the overall grade.
b. If the overall grade is less than 40%, the student fails.
c. If the overall grade is between 40% and 60%, the student gets a C grade.
d. If the overall grade is between 60% and 80%, the student gets a B grade.
e. If the overall grade is above 80%, the student gets an A grade.

Display the overall grade along with the grade letter to the user.

Code:

```
# take input from the user for their grades in five subjects
subject1 = float(input("Enter your grade for subject 1: "))
subject2 = float(input("Enter your grade for subject 2: "))
subject3 = float(input("Enter your grade for subject 3: "))
subject4 = float(input("Enter your grade for subject 4: "))
subject5 = float(input("Enter your grade for subject 5: "))

# calculate the overall grade
overall_grade = (subject1 + subject2 + subject3 + subject4 + subject5) / 5

# assign the grade letter based on the overall grade
if overall_grade < 40:
    grade_letter = "F"
elif overall_grade < 60:
    grade_letter = "C"
elif overall_grade < 80:
    grade_letter = "B"
else:
    grade_letter = "A"

print("Student Grade Sheet")
print("********************")
# display the overall grade and grade letter
print("Overall grade is:", overall_grade)
print("Grade letter is:", grade_letter)
```

OUTPUT

```
Enter your grade for subject 1: 68
Enter your grade for subject 2: 72
Enter your grade for subject 3: 91
Enter your grade for subject 4: 87
Enter your grade for subject 5: 89
Student Grade Sheet
********************
Overall grade is: 81.4
Grade letter is: A
```

7. Write a python program to demonstrate the concept of keyword argument function and default argument function.

```
# Keyword argument function
def greet(name, greeting):
```

```
    print(f"{greeting}, {name}!")
# Default argument function
def calculate_total(price, tax_rate=0.1):
    total = price + (price * tax_rate)
    return total
# Testing keyword argument function
greet(name="Alice", greeting="Hello")
greet(greeting="Hi", name="Bob")
# Testing default argument function
total1 = calculate_total(100)
total2 = calculate_total(100, 0.15)
print(f"Total 1: {total1}")
print(f"Total 2: {total2}")
```

OUTPUT

Hello, Alice!
Hi, Bob!
Total 1: 110.0
Total 2: 115.0

8. . Write a Python program to create a shopping cart for a customer. The program should have at least three functions:

a. add_item(item_name, item_price): This function should add an item and its price to the shopping cart.
b. remove_item(item_name): This function should remove an item from the shopping cart.
c. view_cart(): This function should display the items in the shopping cart and the total cost.

Code

```
# initialize an empty dictionary to store items and their prices

shopping_cart = {}


# function to add an item and its price to the cart

def add_item(item_name, item_price):

    shopping_cart[item_name] = item_price

    print(f"{item_name} added to cart.")
```

```
# function to remove an item from the cart
def remove_item(item_name):
    if item_name in shopping_cart:
        del shopping_cart[item_name]
        print(f"{item_name} removed from cart.")
    else:
        print(f"{item_name} is not in the cart.")

# function to view the items and total cost in the cart
def view_cart():
    if shopping_cart:
        print("Shopping Cart:")
        total_cost = 0
        for item, price in shopping_cart.items():
            print(f"{item}: ${price:.2f}")
            total_cost += price
        print(f"Total cost: ${total_cost:.2f}")
    else:
        print("Your cart is empty.")

# example usage of the functions
add_item("Apples", 1.50)
add_item("Bananas", 0.99)
add_item("Oranges", 1.25)
remove_item("Bananas")
view_cart()
```

OUTPUT

Apples added to cart.

Bananas added to cart.

Oranges added to cart.

Bananas removed from cart.

Shopping Cart:

Apples: $1.50

Oranges: $1.25

Total cost: $2.75

9. Write a Python **module** named "inventory_mgmt" which contains functions for managing an inventory of items. The module should have functions for adding items to the inventory, removing items from the inventory, and updating the quantity of items in the inventory. The functions should take inputs like item name, item quantity, item price, etc., and store the inventory details in a list.

Code:

```
inventory = []


def add_item(name, quantity, price):
    inventory.append({"name": name, "quantity": quantity, "price": price})


def remove_item(name):
    for item in inventory:
        if item["name"] == name:
            inventory.remove(item)
            break


def update_quantity(name, quantity):
    for item in inventory:
```

```
        if item["name"] == name:
            item["quantity"] = quantity
            break
```

#The above module must be used in other python Script

```
import inventory_mgmt

inventory_mgmt.add_item("apple", 10, 1.99)
inventory_mgmt.add_item("banana", 5, 0.99)

inventory_mgmt.remove_item("apple")

inventory_mgmt.update_quantity("banana", 10)

print(inventory_mgmt.inventory)
```

OUTPUT

[{'name': 'banana', 'quantity': 10, 'price': 0.99}]

10. Create a package named product_utils which contains modules product_title, product_description, and product_price. The product_title module should have a function for capitalizing the first letter of each word in the product title. The product_description module should have a function for splitting a product description into bullet points. The product_price module should have functions for formatting the price with appropriate currency symbols and for calculating the discounted price.

Code:

Structure

```
product_utils/
    __init__.py
    product_title.py
```

product_description.py

product_price.py

Code:

#create a folder **product_utils** and place all the below files in this folder.

#Create an empty file

 # __init__.py

#product_title.py

```
def capitalize_title(title):
    return ' '.join([word.capitalize() for word in title.split()])
```

#Product_description.py

```
def create_bullet_points(description):
    return description.split('\n')
```

#Product_price.py

```
def format_price(price, currency='USD'):
    if currency == 'USD':
        return '$' + str(price)
    elif currency == 'EUR':
        return '€' + str(price)
    # Add support for other currencies here
    else:
        return str(price)

def calculate_discounted_price(price, discount):
    return price * (1 - discount)
```

#After creating these modules, you can use them in your code as follows:

Create a python script **mainprg.py**

```
from product_utils import product_title, product_description, product_price

# Example usage
title = "awesome product"
capitalized_title = product_title.capitalize_title(title)
print(capitalized_title)

description = "Features:\n- High quality\n- Durable\n- Stylish"
bullet_points = product_description.format_bullet_points(description)
print(bullet_points)

price = 100.0
formatted_price = product_price.format_price(price)
print(formatted_price)

discounted_price = product_price.calculate_discounted_price(price, 0.2)
print(product_price.format_price(discounted_price))
```

OUTPUT

Awesome Product

['Features:', '- High quality', '- Durable', '- Stylish']

$100.0

$80.0

11. Write a python program to demonstrate the concept of File Operation .

Write a program that reads a file named input.txt, containing multiple lines of text. For

each line in the file, the program should count the number of vowels (a, e, i, o, u) in the line and write the result to a new file named output.txt. Each line in the output file should be in the following format: "Line X: Y vowels, where X is the line number (starting from 1) and Y is the number of vowels in the line.

#Save the given data as Input.txt in the current directory.
India
Australia
Africa
Srilanka
Pakistan
America
United Kingdom
Russia

create a separate input.txt file with some content and store it in your current working directory

```
def count_vowels(line):
    vowels = "aeiouAEIOU"
    return sum(1 for char in line if char in vowels)

def main():
    with open("input.txt", "r") as input_file:
        lines = input_file.readlines()

    with open("output.txt", "w") as output_file:
        for i, line in enumerate(lines, start=1):
            vowel_count = count_vowels(line)
            output_file.write(f"Line {i}: {vowel_count} vowels\n")

# Call the main function directly
main()
```

open a file output.txt to see the result
Output.txt

OUTPUT

Line 1: 3 vowels
Line 2: 5 vowels
Line 3: 3 vowels
Line 4: 3 vowels

Line 5: 3 vowels
Line 6: 4 vowels
Line 7: 5 vowels
Line 8: 3 vowels

12. Write a python program to demonstrate the concept of Regular Expression

    Write a Python program that uses regular expressions to extract this information from the text data.The text data consists of a long string that includes multiple phone numbers,
    email addresses, and postal addresses. Your program should extract the following information from the text data:

    a. All phone numbers, formatted as (+XX) XXXXXXXXXX
    b. All email addresses, which follow the standard format of username@domain.com
    c. All postal addresses, which consist of a street address, city, state, and ZIP code,
    with each part separated by a comma

Code

```
import re

def extract_information(text):
    phone_numbers = re.findall(r'\(\+\d{2}\) \d{10}', text)
    email_addresses = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', text)
    postal_addresses = re.findall(r'\b\d+ .+?, .+?, .+?, \d{5}\b', text)
    return phone_numbers, email_addresses, postal_addresses

text_data = """
Sample text with (+91) 1234567891 phone numbers and email@example.com email addresses.
Also, 123 Main St, City, State, 12345 is a sample postal address.
"""

phone_numbers, email_addresses, postal_addresses = extract_information(text_data)

print("Phone Numbers:")
for number in phone_numbers:
    print(number)

print("\nEmail Addresses:")
for email in email_addresses:
```

```
    print(email)

print("\nPostal Addresses:")
for address in postal_addresses:
    print(address)
```

OUTPUT

Phone Numbers:
(+91) 1234567891

Email Addresses:
email@example.com

Postal Addresses:
123 Main St, City, State, 12345

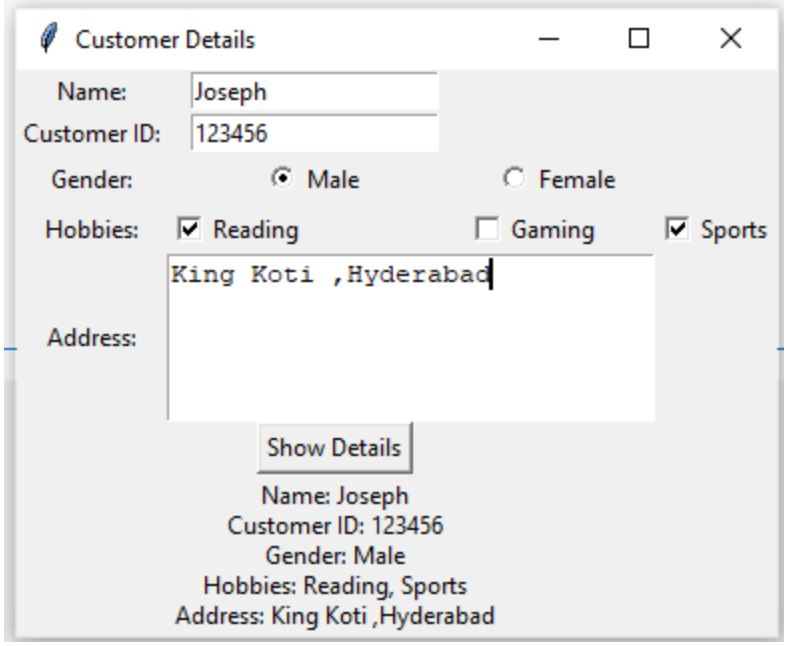
13.
Write a python GUIprogram to display the Customer details like Name, Customer ID, Gender, Hobbies, Address using Label, Button ,Text, Entry ,Radio, CheckButton and Tkinter widgets.

Code

```
import tkinter as tk

def show_details():
    name = name_entry.get()
    customer_id = id_entry.get()
    gender = gender_var.get()
    hobbies = ", ".join(hobbies_list[i] for i, var in enumerate(hobby_vars) if var.get())
    address = address_text.get("1.0", tk.END).strip()

    details_label.config(text=f"Name: {name}\nCustomer ID: {customer_id}\nGender:
{gender}\nHobbies: {hobbies}\nAddress: {address}")

root = tk.Tk()
root.title("Customer Details")

# Labels and Entries
tk.Label(root, text="Name:").grid(row=0, column=0)
```

```
name_entry = tk.Entry(root)
name_entry.grid(row=0, column=1)

tk.Label(root, text="Customer ID:").grid(row=1, column=0)
id_entry = tk.Entry(root)
id_entry.grid(row=1, column=1)

tk.Label(root, text="Gender:").grid(row=2, column=0)
gender_var = tk.StringVar()
male_radio = tk.Radiobutton(root, text="Male", variable=gender_var, value="Male")
male_radio.grid(row=2, column=1)
female_radio = tk.Radiobutton(root, text="Female", variable=gender_var, value="Female")
female_radio.grid(row=2, column=2)

tk.Label(root, text="Hobbies:").grid(row=3, column=0)
hobbies_list = ["Reading", "Gaming", "Sports"]
hobby_vars = [tk.IntVar() for _ in hobbies_list]
for i, hobby in enumerate(hobbies_list):
    tk.Checkbutton(root, text=hobby, variable=hobby_vars[i]).grid(row=3, column=i+1, sticky="w")

tk.Label(root, text="Address:").grid(row=4, column=0)
address_text = tk.Text(root, height=5, width=30)
address_text.grid(row=4, column=1, columnspan=2)

# Button to show details
show_button = tk.Button(root, text="Show Details", command=show_details)
show_button.grid(row=5, column=0, columnspan=3)

# Label to display details
details_label = tk.Label(root, text="")
details_label.grid(row=6, column=0, columnspan=3)

root.mainloop()
```

OUTPUT



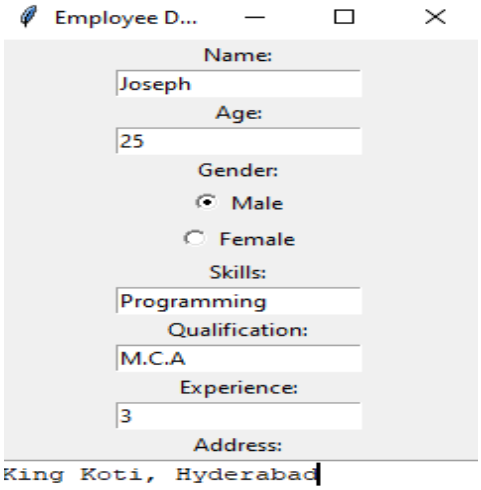
14. Write a python program to display the employee details like Name, age, gender, skills, Qualification, Experience, address using pack() layout method. Add a button control to display the details.

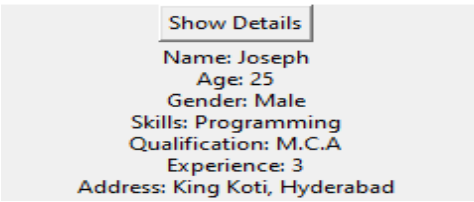
Code:

```
import tkinter as tk

def display_details():
    name = name_entry.get()
    age = age_entry.get()
    gender = gender_var.get()
    skills = skills_entry.get()
    qualification = qualification_entry.get()
    experience = experience_entry.get()
    address = address_text.get("1.0", tk.END).strip()

    details_label.config(text=f"Name: {name}\nAge: {age}\nGender: {gender}\nSkills: {skills}\nQualification: {qualification}\nExperience: {experience}\nAddress: {address}")

root = tk.Tk()
root.title("Employee Details")
```

```
# Name
tk.Label(root, text="Name:").pack()
name_entry = tk.Entry(root)
name_entry.pack()

# Age
tk.Label(root, text="Age:").pack()
age_entry = tk.Entry(root)
age_entry.pack()

# Gender
gender_var = tk.StringVar()
tk.Label(root, text="Gender:").pack()
male_radio = tk.Radiobutton(root, text="Male", variable=gender_var, value="Male")
male_radio.pack()
female_radio = tk.Radiobutton(root, text="Female", variable=gender_var, value="Female")
female_radio.pack()

# Skills
tk.Label(root, text="Skills:").pack()
skills_entry = tk.Entry(root)
skills_entry.pack()

# Qualification
tk.Label(root, text="Qualification:").pack()
qualification_entry = tk.Entry(root)
qualification_entry.pack()

# Experience
tk.Label(root, text="Experience:").pack()
experience_entry = tk.Entry(root)
experience_entry.pack()

# Address
tk.Label(root, text="Address:").pack()
address_text = tk.Text(root, height=5, width=30)
address_text.pack()

# Button to display details
show_button = tk.Button(root, text="Show Details", command=display_details)
show_button.pack()

# Label to display details
details_label = tk.Label(root, text="")
```

```
details_label.pack()

root.mainloop()
```





15. Design a GUI program that displays a text editor with a menu bar. The program should have a menu with options to create a new file, open an existing file, save the current file, and exit the program. The program should also have a menu with options to cut, copy, paste, and select all text in the editor.

```
import tkinter as tk
from tkinter import messagebox
from tkinter import filedialog

def new_file():
    text.delete(1.0, tk.END)
```

```
def open_file():
    file_path = filedialog.askopenfilename(filetypes=[("Text files", "*.txt"), ("All files", "*.*")])
    if file_path:
        with open(file_path, "r") as file:
            text.delete(1.0, tk.END)
            text.insert(1.0, file.read())

def save_file():
    file_path = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Text files",
"*.txt"), ("All files", "*.*")])
    if file_path:
        with open(file_path, "w") as file:
            file.write(text.get(1.0, tk.END))

def cut_text():
    text.event_generate("<<Cut>>")

def copy_text():
    text.event_generate("<<Copy>>")

def paste_text():
    text.event_generate("<<Paste>>")

def select_all_text():
    text.tag_add("sel", "1.0", "end")

root = tk.Tk()
root.title("Simple Text Editor")

text = tk.Text(root, wrap="word")
text.pack(expand=True, fill="both")

# Menu
menu_bar = tk.Menu(root)
root.config(menu=menu_bar)

file_menu = tk.Menu(menu_bar, tearoff=False)
menu_bar.add_cascade(label="File", menu=file_menu)
file_menu.add_command(label="New", command=new_file)
file_menu.add_command(label="Open", command=open_file)
file_menu.add_command(label="Save", command=save_file)
file_menu.add_separator()
file_menu.add_command(label="Exit", command=root.destroy)
```

```
edit_menu = tk.Menu(menu_bar, tearoff=False)
menu_bar.add_cascade(label="Edit", menu=edit_menu)
edit_menu.add_command(label="Cut", command=cut_text)
edit_menu.add_command(label="Copy", command=copy_text)
edit_menu.add_command(label="Paste", command=paste_text)
edit_menu.add_separator()
edit_menu.add_command(label="Select All", command=select_all_text)

root.mainloop()
```

Output

Simple Text Editor
File
Edit
Cut
Copy
Paste
Select All